

DOCUMENTO DE PROPUESTA Y ESTADO DE DESARROLLO

EmerRed

Sistema nacional de alerta, contacto y coordinación para catástrofes.
Funciona con o sin Internet. Conecta afectados, voluntarios y fuentes oficiales.

Audiencia	Alcance del documento
Público general y jurado	Qué problema resuelve, cómo se activa sin instalar app y por qué importa para Colombia.
Persona técnica	Arquitectura, stack, esquemas de datos, endpoints, agente OmniSynth y fallbacks de conectividad.
Equipo de producto	Estado real del código, huecos de la demostración y pasos para llevarlo a producción gubernamental.

Tabla 1. Para quién es este documento y qué cubre cada lectura.

Idea central: cuando ocurre un desastre, el celular suele perder datos o voz, pero el **Cell Broadcast Service (CBS)** sigue llegando a todos los teléfonos de una zona sin app, sin internet y sin contactos previos. EmerRed propone usar ese canal oficial para abrir una ayuda puntual (PWA o USSD), registrar ubicación una sola vez y coordinar auxilio con un agente de IA. La app móvil de demostración (fork de bitchat: APK Android y app iOS) es solo la simulación del hackathon.

1. Explicación para el público general

EmerRed no pide que la gente instale una aplicación en medio del pánico. La propuesta real para el Gobierno de Colombia es activar el sistema desde los mecanismos oficiales de alerta (UNGRD, gobernaciones, CMGRD) y las operadoras móviles. El usuario solo ve una alerta en su teléfono, toca un enlace o marca un código corto, y puede pedir ayuda.

1.1 El problema en lenguaje simple

Situación hoy	Qué pasa con las personas	Qué propone EmerRed
Se cae internet o la señal de datos	No pueden abrir WhatsApp ni apps de emergencia	La alerta llega por CBS, que no depende de internet
Hay que instalar una app	En pánico no hay tiempo, espacio ni datos	No se instala nada: PWA en el navegador o USSD *119#
Nadie sabe dónde están los afectados	Las salas de mando deciden casi a ciegas	Un toque autoriza GPS puntual y genera un reporte
Zonas sin torre celular	Quedan invisibles	Red mesh Bluetooth y cola offline hasta recuperar cobertura
Demasiada información suelta	Voluntarios y autoridades se descoordinan	La IA prioriza, agrupa y sugiere recursos

Tabla 2. Problema cotidiano y respuesta de EmerRed.

1.2 Cómo lo usaría una persona afectada

Paso	Lo que ve la persona	Lo que ocurre por detrás
1	Alerta oficial en la pantalla bloqueada	UNGRD / CMGRD dispara CBS por zona
2	Toca el enlace o marca *119#	Se abre PWA o menú USSD, sin instalar app
3	Autoriza ubicación una sola vez	Se captura lat/long puntual, no hay rastreo continuo
4	Responde 3 o 4 preguntas simples	El agente hace triaje y asigna prioridad
5	Recibe instrucciones claras	El reporte llega al dashboard de autoridades y voluntarios
6	Si no hay señal, el teléfono guarda el pedido	Mesh o buffer offline lo envían al recuperar cobertura

Tabla 3. Recorrido de la persona afectada, de la alerta a la ayuda.

Mensaje para el jurado y para ciudadanía: **la red que no se cae cuando todo lo demás falla**. CBS ya existe en los celulares colombianos. EmerRed lo convierte en un puente accionable hacia auxilio, no solo en un aviso de texto.

2. Explicación para una persona técnica

El repositorio es un monorepo. La demostración del hackathon vive en una aplicación móvil nativa: APK para Android y app para iOS, fork de bitchat, usada para ensayar mesh Bluetooth y el flujo de alerta. La propuesta de producción es un sistema gubernamental: CBS de operadoras, PWA/USSD sin instalación, API REST en Railway, MongoDB Atlas, dashboard de operadores y el agente OmniSynth (Python) como cerebro de triaje, inteligencia situacional y comunicación multicanal.

2.1 Capas del sistema

Capa	Rol	En la demo	En producción gubernamental
Activación	Disparo masivo georreferenciado	Botón que simula alerta CBS	CBS real vía operadoras (Claro, Tigo, Movistar, WOM) disparado por UNGRD/CMGRD
Canal de usuario	Pedir ayuda sin fricción	App nativa (fork de bitchat): APK Android + iOS	PWA (emerred.gov.co) + fallback USSD *119# + SMS
Captura	Ubicación y contexto del afectado	GPS puntual en la app nativa (Android/iOS)	Geolocation web / red de operadora / mesh BLE hacia gateway
API	Persistencia y auth	Express + Mongoose en Railway	Misma API o evolución con índices geo y roles institucionales
Agente IA	Triage, síntesis, coordinación	OmniSynth CLI + cliente de emergencia	Servicio de triaje, informe situacional y despacho CBS/PWA/USSD
Mando	Ver y asignar casos	Dashboard React (parcialmente mock)	Consolas UNGRD, gobernaciones y CMGRD/Bomberos

Tabla 4. Arquitectura lógica: demo vs. sistema real.

2.2 Stack actual del monorepo

Componente	Tecnología	Ubicación	Estado
API REST	Node 20, Express 4, Mongoose 8, Joi, Swagger, JWT, bcryptjs	backend/	Desplegada
Base de datos	MongoDB Atlas (producción) / local opcional	MONGODB_URI	Conectada
Auth	POST /auth/login, JWT 8h, roles admin/operator/viewer	backend/src/{models,routes,middlewares}	Operativo

Componente	Tecnología	Ubicación	Estado
Agente OmniSynth	Python, Gemini, httpx, buffer offline, RSS/web	agente/	No conectado al backend live
Dashboard operadores	React 19, Vite, TypeScript, Tailwind, Recharts, Pigeon Maps	dashboard/ · Vercel	Desplegado (datos mock)
App demo	Fork de bitchat: APK Android + app iOS (CBS, GPS, chat, mesh)	app nativa / fork bitchat	Simulación de hackathon
Deploy API	Dockerfile + railway.json (contexto raíz, solo backend/)	Railway	https://emerred-production.u p.railway.app
Deploy dashboard	Vite static build en Vercel	dashboard/	Producción en Vercel

Tabla 5. Inventario técnico del repositorio.

2.3 Dashboard de operadores (Vercel)

El panel de administración vive en **dashboard/** y está desplegado en **Vercel** como sitio estático (build de Vite). Es la consola que vería un operador UNGRD/CMGRD en la demo: login, resumen, alertas, mapa y asignación de voluntarios. Hoy consume datos mock en el cliente; el cableado a la API de Railway es el siguiente paso para que el mapa muestre afectados reales.

Ítem	Detalle técnico
Plataforma de deploy	Vercel (front estático). Build: bun/npm run build → tsc -b && vite build
Runtime local	Bun + Vite dev server en http://localhost:5173
Framework	React 19.2 + TypeScript 5.6 + Vite 8
UI	Tailwind CSS v4 (@tailwindcss/vite), Lucide React, Recharts, Pigeon Maps
Arquitectura	Clean Architecture: domain/ (tipos), data/ (adaptadores), presentation/ (UI + hooks)
Alias	@ → dashboard/src (vite.config.ts)
Auth actual	Mock en data/auth.ts. Cookie emerred_token. Credenciales: admin@emerred.co / admin123
Sesión	useIdleTimeout: cierra sesión a los 10 minutos de inactividad
Datos actuales	api.ts genera ~80 reportes mock (Bogotá) y 1 alerta activa; no llama a Railway
Tabs	Resumen, Alertas (broadcast CBS simulado), Mapa (pines), Reportes (asignar voluntario)

Tabla 6. Ficha técnica del dashboard desplegado en Vercel.

Módulo	Archivo	Contrato / comportamiento
Tipos	src/domain/types.ts	Report, ActiveAlert, Stats, Priority (critica alta media baja)
API mock	src/data/api.ts	getReports, getActiveAlerts, getStats, assignReport, broadcastAlert
Auth mock	src/data/auth.ts	login() / getAuthToken() / cookie SameSite=Strict. Pendiente POST /auth/login
Orquestación	src/presentation/Dashboard.tsx	Tabs + Suspense data + logout
Mapa	src/presentation/components/map/HeatMap.tsx	Pigeon Maps, pines clickeables, detalle de incidente
Gráficos	charts/PriorityChart.tsx, CategoryChart.tsx	Recharts: prioridad y categoría
Alertas	alerts/BroadcastForm.tsx	Emite alerta mock (tipo, mensaje, ciudad, radio 5 km)

Módulo	Archivo	Contrato / comportamiento
Reportes	reports/ReportsTable.tsx	Tabla + asignación a MOCK_VOLUNTEERS

Tabla 7. Mapa de módulos del dashboard.

Tipo (dashboard)	Campos	Relación con la API Railway
Report	id, lat, lon, accuracy, prioridad, categoria, mensaje, asignado, direccion, ciudad, creado	Más rico que Afectado. Falta mapear lat/long, celular, dBm y mesh → prioridad/categoría
ActiveAlert	id, active, tipo, mensaje, ciudad, radio	No existe aún en backend (CBS/alertas oficiales)
Stats	total, criticas, altas, medias, bajas, asignados, porCategoría	Se puede derivar en cliente o con un GET /stats futuro
AuthResult	token, user {id, email, name}	Compatible con POST /auth/login de la API (JWT 8h)

Tabla 8. Esquemas TypeScript del dashboard frente a la API en Railway.

Para que el deploy de Vercel deje de ser una maqueta: (1) VITE_API_URL=https://emerred-production.up.railway.app; (2) auth.ts → POST /auth/login con Authorization Bearer; (3) api.ts → GET/POST /afectados; (4) CORS_ORIGIN en Railway apuntando al dominio Vercel.

2.4 Esquema Afectado (API actual)

El modelo persistido hoy es deliberadamente mínimo: coordenadas, celular, potencia de red y bandera de mesh. El agente OmniSynth ya modela un payload más rico (tipo de red, operador, batería, metadatos) que todavía no está 1:1 con la API.

Campo	Tipo	Regla	Para qué sirve
_id	ObjectId	Autogenerado	Identidad del reporte
lat	Number	-90 a 90, requerido	Ubicación del afectado
long	Number	-180 a 180, requerido	Ubicación del afectado
numero_celular	Number (entero)	9–10 dígitos, único	Contacto y búsqueda rápida
potencia_red_movil	Number (entero)	-150 a 0 dBm	Calidad de señal / vulnerabilidad
conecccion_mesh	Boolean	Requerido, default false	Indica si llegó por red mallada
createdAt / updatedAt	Date	Timestamps Mongoose	Orden y auditoría

Tabla 9. Esquema MongoDB Afectado (backend/src/models/Afectado.js).

2.5 Esquema User y autenticación

Campo	Tipo	Regla	Notas
email	String	Único, lowercase, formato email	Login institucional
passwordHash	String	bcrypt 12 rounds, select:false	Nunca se expone en JSON
name	String	2–100 caracteres	Nombre visible en dashboard
role	Enum	admin operator viewer	Control de acceso por rol
createdAt / updatedAt	Date	Automático	Auditoría

Tabla 10. Esquema User. Seed al arrancar: admin@gmail.com / 123456 / admin.

2.6 Esquema de telemetría del agente (más amplio que la API)

Campo agente (Pydantic)	Equivalente API	Observación
lat / long	lat / long	Alineado
numero_celular (string E.164)	numero_celular (Number)	Hay que unificar formato (string vs entero)
potencia_red_movil_dbm + tipo_red + operador	potencia_red_movil	La API solo guarda el entero dBm
coneccion_mesh	coneccion_mesh	Alineado (typo histórico conservado)
nivel_bateria	—	Aún no existe en backend
calidad_red	—	El agente la estima; la API no la persiste
timestamp / metadatos	createdAt	El agente envía payload anidado que la API aún no acepta 1:1

Tabla 11. Brecha de esquema entre OmniSynth y la API Express.

2.7 Endpoints de la API en producción

Método	Ruta	Auth	Descripción
POST	/afectados	Público	Crea un afectado
GET	/afectados	Público	Lista todos (createdAt desc)
GET	/afectados/:id	Público	Obtiene por ObjectId
GET	/afectadoPorCelular/:numero_celular	Público	Búsqueda por celular
PUT	/afectados/:id	Público	Actualiza registro
DELETE	/afectados/:id	Público	Elimina registro
POST	/auth/login	Público	Devuelve JWT 8h + user
POST	/auth/register	Admin	Crea usuario
GET	/auth/me	JWT	Usuario actual
POST	/auth/refresh	JWT	Renueva token
GET	/health	Público	Health check
GET	/api-docs	Público	Swagger UI (server Railway primero)

Tabla 12. Superficie HTTP actual. Base: <https://emerred-production.up.railway.app>

2.8 Fallbacks de conectividad

Nivel	Condición	Comportamiento previsto	Implementado hoy
1. Celular / datos	Hay 4G/5G o Wi-Fi	POST directo a /afectados	Sí, API + cliente httpx del agente
2. CBS	Hay radio celular aunque no haya datos	Alerta masiva + deep link / USSD	Solo simulado en app/README
3. Mesh BLE	No hay torre, hay vecinos o gateway	Salto entre nodos hasta un gateway con internet	Campo conexion_mesh + simulación UI
4. Offline buffer	No hay ningún canal	Guarda JSON local y hace flush al recuperar red	Sí, en agente/emergency_client.py
5. USSD / SMS	Teléfono básico o sin smartphone	Menú *119# o SMS corto estructurado	No implementado

Tabla 13. Cascada de conectividad (de más a menos cobertura).

2.9 El agente OmniSynth en este diseño

OmniSynth ya no se presenta solo como investigador web. En EmerRed es el mediador: recibe telemetría, reintenta contra la API, guarda cola offline y, en la visión de producción, hace triaje, inteligencia situacional y elección de canal (CBS, PWA, USSD).

Capacidad	Qué hace	Código actual	Falta para gobierno
Cliente de emergencia	Valida payload, reintenta, buffer 202, flush	agente/synth/tools/emergency_client.py	Apuntar EMERRED_API_URL a Railway y alinear JSON plano de la API
Investigación / síntesis	Planifica búsquedas, scrapea, sintetiza con Gemini	agente/synth/agent.py + prompts.py	Modo informe táctico UNGRD (clima, hospitales, rutas)
Triage	Prioridad + instrucciones + fin de entrevista	Contrato descrito en README; no hay POST /triage en backend actual	Endpoint y prompts de emergencia
CBS	Disparo geofenced y confirmación de lectura	No existe módulo	cbs_integration.py + sandbox de operadoras
Coordinación	Matching voluntario—caso y ETA	Dashboard mock assignReport	POST /assign real + reglas de despacho

Tabla 14. Mapa de capacidades del agente vs. lo que falta.

3. Relatorio del estado actual de desarrollo

Lo que está en producción hoy es la API de afectados y autenticación. El resto del ecosistema (app demo, dashboard, agente, CBS, mesh real) está en distinto grado de madurez. La siguiente tabla es el semáforo del monorepo al momento de este informe.

Módulo	Estado	Evidencia	Limitación
API Afectados CRUD	Listo	backend/src + Railway + Swagger	Rutas de afectados aún públicas; payload más simple que el del agente
Auth JWT + seed admin	Listo	login/register/me/refresh; JWT_SECRET en Railway	Dashboard todavía usa token fake y credenciales distintas
Health + CORS + graceful shutdown	Listo	/health, server.js	—
Docker / Railway monorepo	Listo	railway.json + backend/Dockerfile + .dockerignore	Variables JWT deben setearse a mano en el dashboard
Agente emergency client	Parcial	Tests de buffer/offline en tests/test_emergency.py	URL por defecto api.emerred.org.co; payload anidado no coincide con API
Agente research Gemini	Parcial	CLI research/urls/rss	No está cableado al dashboard ni a sala de mando
Dashboard operadores (Vercel)	Parcial	SPA Vite/React desplegada en Vercel; tabs Resumen/Alertas/Mapa/Reportes	Datos y login aún mock; falta VITE_API_URL → Railway y CORS_ORIGIN
App nativa (fork de bitchat)	Demo	APK Android + app iOS para CBS, GPS, chat y mesh	Es simulación de hackathon, no el producto gubernamental
CBS / operadoras	Pendiente	Concepto y tablas de este documento	Requiere acuerdos UNGRD–MinTIC–operadoras
Mesh BLE real	Pendiente	Solo flag + pantalla simulada	El mesh de producción requiere gateways fijos; la app nativa solo demuestra el salto BLE

Módulo	Estado	Evidencia	Limitación
PWA / USSD *119#	Pendiente	Propuesta de producción	No hay front web de afectados ni gateway USSD
POST /triage y /assign	Pendiente	Especificados en README raíz	No existen en el backend Express actual

Tabla 15. Semáforo de desarrollo. Verde = listo, ámbar = parcial, rojo = pendiente.

3.1 Lo que ya se puede demostrar hoy con la API

- Health check: GET <https://emerred-production.up.railway.app/health>
- Swagger: <https://emerred-production.up.railway.app/api-docs> (servidor de producción primero)
- Login: POST /auth/login con {"email":"admin@gmail.com","password":"123456"}
- CRUD de afectados y búsqueda por celular
- CLI del agente: `python -m synth emergency --json '...'` (si se apunta a Railway y se aplanan el JSON)

4. Qué falta para completar la demostración (hackathon)

La demo no necesita CBS real ni contratos con operadoras. Necesita un relato continuo en 3–5 minutos: alerta → autorización → reporte → mapa → voluntario → fallback sin señal. Abajo está el backlog mínimo, ordenado por impacto en el pitch.

#	Entrega de demo	Por qué importa en el pitch	Esfuerzo relativo
1	Conectar <code>dashboard.login()</code> a POST /auth/login de Railway y guardar JWT real	Cierra el círculo “hay usuarios institucionales”	Bajo
2	Conectar dashboard a GET/POST /afectados (dejar de usar <code>MOCK_REPORTS</code>)	El mapa muestra datos vivos, no 80 puntos inventados	Medio
3	Alinear el cliente del agente al JSON plano de la API y a la URL de Railway	Se ve el buffer offline y el flush en vivo	Medio
4	App nativa (APK/iOS): simular CBS + GPS puntual + envío a /afectados	El jurado entiende “sin app en producción, con app solo para ensayar”	Medio
5	Mock visual de mesh (nodos y saltos) + flag <code>conexcion_mesh=true</code>	Explica el fallback cuando no hay torre	Bajo
6	POST /triage mínimo (reglas o Gemini) que devuelva prioridad + markdown	Muestra al agente como mediador, no solo como CRUD	Medio / alto
7	Guion de 3 minutos + 1 informe OmniSynth de “situación” exportado a HTML	Sala de mando: del dato crudo a la decisión	Bajo

Tabla 16. Backlog mínimo para cerrar la demostración.

4.1 Criterio de “demo lista”

Escena	Acción en escena	Prueba de que funciona
Alerta	Botón “Simular CBS” en el celular de demo	Pantalla de alerta con deep link / CTA
Pedido de ayuda	Autorizar GPS y enviar reporte	201 en /afectados y punto nuevo en dashboard
Login operador	Entrar con <code>admin@gmail.com</code>	JWT 8h y /auth/me coherente
Sin red	Apagar datos y reenviar desde el agente	Código 202 + archivo de buffer + flush posterior
Mesh	Marcar <code>conexcion_mesh=true</code>	El dashboard distingue reportes mesh vs. celulares

Tabla 17. Checklist de aceptación de la demo.

5. Pasos para desarrollarlo en la realidad

Pasar de prototipo a sistema nacional implica instituciones, no solo código. CBS es un servicio de las operadoras bajo marco de alerta pública; la PWA o el USSD deben ser canales oficiales; los gateways mesh deben vivir en estaciones de bomberos, alcaldías y Cruz Roja. La IA no sustituye al CMGRD: prioriza y explica.

5.1 Actores y responsabilidades

Actor	Responsabilidad	Interfaz con EmerRed
UNGRD / CMGRD	Decide cuándo y dónde alertar; opera la sala de mando	Dashboard nacional + disparo CBS
MinTIC / ANE	Marco de CBS, numeración USSD, privacidad y espectro	Habilitación regulatoria
Operadoras móviles	Difunden CBS y pueden ofrecer USSD/SMS de emergencia	API/sandbox CBS por polígono o celda
Salud / Cruz Roja / Bomberos	Atienden y confirman casos	App o PWA de voluntario + gateways mesh
Gobernaciones y alcaldías	Recursos territoriales y evacuación	Dashboard filtrado por departamento / municipio
Ciudadanía	Autoriza GPS puntual y pide ayuda	CBS nativo + PWA o *119#, sin instalar app

Tabla 18. Gobernanza del sistema real.

5.2 Hoja de ruta

Fase	Plazo orientativo	Entregable	Dependencia no técnica
Piloto ciudad	0–3 meses	PWA + API endurecida + 1 sandbox CBS + 1 estación con gateway	Convenio UNGRD + una operadora + un cuerpo de bomberos
Regional	3–6 meses	Dashboards de gobernación, USSD *119#, mesh en 5 departamentos	Gobernaciones, Cruz Roja, Defensa Civil
Nacional	6–12 meses	4 operadoras, espejo en radio/TV, auditoría y SLA	MinTIC, ANE, RTVC, UNGRD
Ecosistema	12–18 meses	API pública GeoJSON, estándar de reporte, apps de terceros	Política de datos abiertos de emergencia

Tabla 19. Fases de implementación real.

5.3 Trabajo técnico de producción (después del hackathon)

Frente	Tarea concreta	Prioridad
Contrato de datos	Unificar celular como string, aceptar payload anidado o versionar /v2/afectados	Alta
Seguridad	Proteger mutaciones de afectados con JWT/roles; secretos rotables; rate limit	Alta
Geo	Índice 2dsphere y GET /afectados/zona?lat&long;&r;=	Alta
Triaje	POST /triage con modelo + fallback por reglas si no hay LLM	Alta
CBS	Módulo de disparo, recodificación a 93 caracteres, reenvío si no hay apertura del link	Alta
PWA afectados	Service worker, Web Geolocation, Web Push, modo offline	Alta
USSD/SMS	Gateway *119# para feature phones	Media
Mesh	Firmware/gateway BLE-LoRa en estaciones fijas; la app nativa (fork de bitchat) queda como cliente de campo	Media
Observabilidad	Métricas de alcance CBS, tiempo a primer auxilio, flush de buffer	Media

Frente	Tarea concreta	Prioridad
Privacidad	Hash de celular en analítica, consentimiento puntual, retención limitada	Alta

Tabla 20. Backlog de producción, independiente de la demo.

6. Lectura rápida para el jurado

Pregunta del jurado	Respuesta EmerRed
¿La gente tiene que instalar una app?	No en producción. CBS + PWA o USSD. La app nativa (fork de bitchat, APK Android e iOS) es solo el laboratorio del hackathon.
¿Qué hay construido de verdad?	API en Railway (CRUD + JWT + Swagger), dashboard de operadores en Vercel (UI viva, datos mock) y agente con buffer offline.
¿Qué es simulación?	CBS, mesh BLE, chat de triaje extremo a extremo, dashboard con datos mock y USSD.
¿Por qué involucrar operadoras?	Solo ellas pueden disparar CBS a todas las celdas de un polígono sin internet ni agenda de contactos.
¿Qué hace la IA que no haga un formulario?	Prioriza, explica, agrupa casos, sugiere recursos y elige canal cuando la red es inestable.
¿Cuál es el siguiente paso creíble?	Piloto: una ciudad, una operadora en sandbox CBS, una estación de bomberos como gateway, PWA oficial.

Tabla 21. Preguntas frecuentes del pitch.

Referencias internas del repositorio

- README.md — producto del hackathon y distinción demo vs. producción.
- AGENTS.md — contexto compacto para agentes de IA (stack, endpoints, auth).
- backend/ — API Express desplegada en emerred-production.up.railway.app.
- agente/ — OmniSynth y cliente resiliente de telemetría.
- dashboard/ — consola de operadores en Vercel (React/Vite); datos aún mock.

Este PDF no sustituye el código ni el pitch oral. Fija un lenguaje común: para la ciudadanía, EmerRed es la alerta que se convierte en ayuda; para ingeniería, es una cascada CBS → PWA/USSD → API → agente → mando; para el equipo, es un mapa honesto de lo listo, lo parcial y lo que aún es promesa.

Documento generado a partir del estado del monorepo EmerRed y de la propuesta de integración gubernamental (CBS, operadoras, PWA/USSD, OmniSynth).